Reproducing Frozen-InceptionV3 Pneumonia Detection on Pediatric Chest X-Rays, and Fine-Tuning Its Top Blocks

Vaiditya Tanwar
Computer Vision Course Project (Milestone 5)

Abstract—We reproduce the chest X-ray experiment of Kermany et al. (Cell, 2018), in which a frozen ImageNet InceptionV3 network is used as a feature extractor and only a new linear softmax layer is trained to classify pediatric chest X-rays as NORMAL or PNEUMONIA. On the official public split (5 232 training, 624 test images) and over five random seeds, our PyTorch reproduction reaches an accuracy of 80.13±0.74%, sensitivity 98.41±0.30%, specificity 49.66±1.94% and ROC-AUC 92.89±0.24%, against 92.8% accuracy and 96.8% AUC reported by the authors. The gap is almost entirely false positives on NORMAL images, and validation accuracy on held-out training images (98.7%) is far above test accuracy, pointing to a train/test distribution shift and an unsuitable 0.5 decision threshold. We then test one change: fine-tuning the last two Inception blocks (Mixed_7b, Mixed_7c) at a small learning rate. Two blocks unfrozen gives AUC TBD% versus TBD% for the frozen model (ΔAUC = TBD, Δspecificity = TBD points). This is a student reproduction study and makes no clinical claims.

Index Terms—chest X-ray, pneumonia, transfer learning, InceptionV3, reproducibility, fine-tuning

I. Introduction

Pneumonia is an acute infection of the lungs and remains the single largest infectious cause of death in children worldwide. According to the World Health Organization, it killed 740 180 children under the age of five in 2019, accounting for 14% of all deaths in that age group [1]. Most of these deaths are preventable or treatable when the disease is identified early, so fast and reliable diagnosis matters.

The chest X-ray (CXR) is the most common imaging test used to confirm pneumonia, but it is not easy to read. Signs such as patchy opacities or consolidation can be subtle, can overlap with other conditions, and look different in small children than in adults. Reading CXRs well requires trained radiologists, and these specialists are scarce in many of the low-resource settings where the pneumonia burden is highest [3]. This motivates research into automated tools that could support (not replace) clinical readers.

Deep convolutional neural networks (CNNs) have shown strong results on medical images. CheXNet, a 121-layer CNN trained on more than 100 000 chest X-rays, was reported to detect pneumonia at a level comparable to practising radiologists on a held-out test set [3]. Because labelled medical datasets are usually small, most such systems rely on transfer learning: a network is first trained on a large natural-image dataset (ImageNet) and then reused for the medical task. Yosinski et al. showed that early CNN layers learn general features (edges, textures) while later layers become specific to the original task, so how much of a network to re-train is an important choice [4]. Raghu et al. further found that for medical images, which differ strongly from natural photos, the benefit of ImageNet features is smaller than often assumed and is concentrated in the lowest layers [5].

Our base paper is Kermany et al. (Cell, 2018) [2]. The authors built a transfer-learning system for retinal OCT scans and then showed it also generalises to classifying pediatric chest X-rays as normal or pneumonia. Their method uses an ImageNet-pretrained InceptionV3 network that is kept completely frozen as a feature extractor; only a new final linear classification layer is trained. They released the dataset (about 5 200 training and 624 test images) publicly [6], which makes the work well suited for reproduction.

Our contribution. We (i) reproduce the frozen-InceptionV3 baseline on the same public dataset and report accuracy, sensitivity, specificity, precision and ROC-AUC over five seeds, (ii) analyse where and why the reproduction falls short of the published numbers, and (iii) test one targeted change, fine-tuning only the last two Inception blocks together with the classifier, with a small ablation over how much of the network is unfrozen.

II. Related Work

Transfer learning for CXR. Kermany et al. [2] use a frozen InceptionV3 [7] pretrained on ImageNet [8]. CheXNet [3] instead fine-tunes an entire DenseNet-121 on the much larger ChestX-ray14 dataset [9]. Our change sits between these two extremes.

Which layers to adapt. Yosinski et al. [4] show that transferability falls with depth, which suggests adapting the top layers first. Raghu et al. [5] show that on medical images the top of large ImageNet models changes the most during fine-tuning and that pretrained features help less than expected.

Generalisation of pneumonia detectors. Zech et al. [10] found that CNN pneumonia detectors performed noticeably worse on X-rays from hospitals not seen in training,

TABLE I
What is trainable in each configuration. Everything below the trainable part is frozen and cached. Learning rate: head 10^{-3}, unfrozen blocks 10^{-4}. *Smaller budget: batch 32, 1 000 steps, 1 seed.

Configuration	Trainable part	Params
0 blocks (paper)	linear head only	4.1K
1 block	Mixed_7c + head	6.08M
2 blocks (ours)	Mixed_7b, Mixed_7c + head	11.13M
Whole network*	all layers + head	21.79M

and that models can exploit site-specific cues. This is relevant to the train/test shift we observe in Section VI.

III. Method

A. Baseline (reproduced)

Each X-ray is resized directly to 299×299, converted to three identical channels and normalised with ImageNet statistics. The ImageNet-pretrained InceptionV3 from torchvision, with its final layer removed, maps each image to a 2048-dimensional pooled feature vector. All of its weights are frozen, so features are computed once and cached, as in the authors' code. A new linear layer ($2048 \to 2$, weights $\sim \mathcal{N}(0, 0.001^2)$, softmax, cross-entropy) is trained with Adam [11] (learning rate 10^{-3}, batch 256 sampled at random, 4 000 steps), matching the authors' settings.

B. Our change: fine-tune the top two Inception blocks

ImageNet features come from colour photographs of objects; chest X-rays are grayscale and their diagnostic cues (opacities, consolidation) look nothing like ImageNet classes. Following [4], [5], the highest blocks are the most task-specific, so we unfreeze only the last two Inception blocks, Mixed_7b and Mixed_7c, initialised from the ImageNet weights and trained at a smaller learning rate (10^{-4}) so they adapt without forgetting. The head keeps learning rate 10^{-3}. Everything else (data, split, preprocessing, optimiser, batch size, steps) is unchanged. Because the lower 16 stages stay frozen, their output ($8 \times 8 \times 1280$) is cached once and only the top of the network is run during training. Table I summarises what is trained in each configuration.

IV. Experimental Setup

Data. We use the Kermany chest X-ray dataset (Mendeley Data, rscbjbr9sj, v2) [6]: pediatric anterior-posterior CXRs from Guangzhou Women and Children's Medical Center. We keep the official split unchanged: training 1 349 NORMAL / 3 883 PNEUMONIA (5 232), test 234 NORMAL / 390 PNEUMONIA (624). The training set is imbalanced roughly 1:3. For model monitoring we hold out a random 10% of the training set as validation (seed-dependent); the test set is never used for any choice.

Metrics. With PNEUMONIA as the positive class and a 0.5 threshold on P(pneumonia): accuracy, sensitivity (recall, $\frac{TP}{TP+FN}$), specificity ($\frac{TN}{TN+FP}$), precision ($\frac{TP}{TP+FP}$), and threshold-free ROC-AUC. All are reported in percent.

Seeds. The reproduction uses 5 seeds (0–4) and reports mean ± standard deviation. In the ablation, the frozen model uses 5 seeds, 2 blocks uses TBD seeds, 1 block and the whole network one seed each, because fine-tuning runs take minutes instead of seconds.

Implementation. PyTorch 2.15 and torchvision 0.29 with the IMAGENET1K_V1 InceptionV3 weights [12], on an Apple M5 Pro laptop GPU (MPS backend). Code is in two notebooks: pneumonia_baseline_reproduction.ipynb and pneumonia_milestone4_finetune.ipynb.

TABLE II
Paper vs. our reproduction of the frozen baseline (test set, %). Ours: mean±std over 5 seeds.

Metric	Kermany et al. [2]†	Ours
Accuracy	92.8	80.13±0.74
Sensitivity	93.2	98.41±0.30
Specificity	90.1	49.66±1.94
Precision	–	76.52±0.69
ROC-AUC	96.8	92.89±0.24

†Only the 92.8% accuracy could be confirmed from secondary sources; the other three values are as quoted in our project brief and could not be checked against the original text.

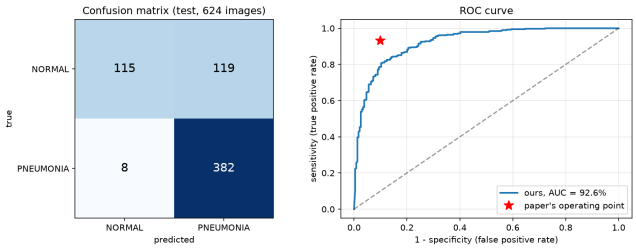


Fig. 1. Frozen baseline, seed 0: confusion matrix on the 624 test images and ROC curve.

V. Results

A. Reproduction of the baseline

Table II compares our reproduction with the paper. Sensitivity is higher than reported, but specificity is about 40 points lower, which pulls accuracy down by about 13 points. AUC, which does not depend on the threshold, is 3.9 points lower. Variation across seeds is small ($\leq$2 points), so the gap is not run-to-run noise.

B. Ablation: how much of the network to fine-tune

Table III reports the ablation. All rows share the same data, preprocessing and evaluation; the 0-block row is re-run inside the Milestone 4 pipeline (cached in float16), so it can differ slightly from Table II. "Train" is accuracy on the 90% training portion.

VI. Failure Analysis

Errors are false positives on NORMAL images. For the frozen baseline (seed 0), 119 of the 127 test errors

TABLE III
Ablation over the number of unfrozen Inception blocks (test set, %, mean±std where more than one seed).

Unfrozen	Seeds	Accuracy	Sensitivity	Specificity	Precision	ROC-AUC	Train acc.	Val. acc.
0 blocks (paper method)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
1 block (Mixed_7c)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
2 blocks (ours)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Whole network*	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

*Smaller training budget (batch 32, 1 000 steps); not directly comparable to the other rows.

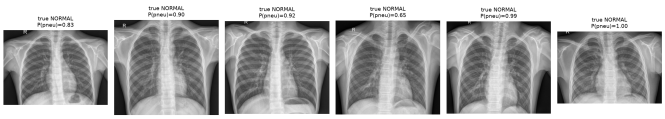


Fig. 2. Misclassified test images of the frozen baseline (seed 0). [Fine-tuned failures figure pending.]

TABLE IV
Pre-registered predictions vs. outcome (2 blocks minus frozen, mean over seeds, percentage points).

Prediction	Observed	Verdict
AUC +1–2	TBD	TBD
Specificity improves most	TBD	TBD
Sensitivity within ∼1	TBD	TBD
Accuracy (not predicted)	TBD	–

are NORMAL images predicted as PNEUMONIA; only 8 pneumonia cases are missed (Fig. 1). With 2 blocks fine-tuned, seed 0 makes TBD false positives and TBD false negatives (baseline in the same pipeline: TBD and TBD). Fig. 2 shows the fine-tuned model's most confident mistakes.

Train/test distribution shift. On held-out training images the frozen model reaches 98.66% validation accuracy, but only 79.65% on the test set (seed 0). Validation and training images come from the same pool, while the official test set was collected and labelled separately. A model that is near-perfect on one and 19 points worse on the other is learning cues that do not carry over; this is the kind of shift Zech et al. [10] warn about.

Threshold. At 0.5 the model is strongly biased towards PNEUMONIA. Moving along the same ROC curve to the paper's operating sensitivity (93.6%, threshold 0.945) gives a specificity of 70.9% instead of 49.7%. The paper does not state its threshold or how the final checkpoint was chosen, so part of the gap may be an operating-point choice rather than a weaker model; the remaining AUC gap (92.9 vs. 96.8) is not explained by the threshold.

Class imbalance. With about three pneumonia images per normal one and no class weighting, the cross-entropy loss is dominated by pneumonia examples, which pushes predicted probabilities upwards and makes false positives the cheap error during training.

Other implementation differences. We use torchvision's InceptionV3 weights rather than the 2015 TensorFlow graph, PIL bilinear resizing rather than TensorFlow's, and our own random validation split. Any of these can move features and therefore results by a few points.

VII. Hypothesis Outcome

Before running Milestone 4 we predicted (Milestone 3) that unfreezing Mixed_7b and Mixed_7c would (a) raise AUC by about 1–2 points, (b) improve specificity more than any other metric, and (c) keep sensitivity within about 1 point. We also stated that the hypothesis is falsified if the AUC gain is $\leq$0 or within seed noise, if AUC rises without a specificity gain, or if training accuracy rises while test performance falls.

TBD An AUC gain counts as real only if it exceeds both 0.5 points and twice the across-seed standard deviation of the baseline AUC. Table III also lets us check the overfitting criterion directly: training accuracy of the fine-tuned models (TBD% for 2 blocks) should be compared with their test accuracy (TBD%). Because only TBD seeds were run for 2 blocks, this conclusion is weaker than the reproduction result.

VIII. Limitations

- One dataset from one hospital and a single fixed test set of 624 images; no external validation.
- Few seeds for the fine-tuned configurations (1–2), and the whole-network row uses a smaller training budget.
- We could not fully verify the paper's reported sensitivity, specificity and AUC, nor its threshold and checkpoint selection.
- A fixed 0.5 threshold is used for all models; no calibration or class weighting.
- No data augmentation was used, to stay faithful to the baseline.
- This is a course reproduction study. The models are not validated for, and must not be used in, any clinical setting.

IX. Conclusion

We reproduced the frozen-InceptionV3 pneumonia classifier of Kermany et al. and obtained a high sensitivity but a much lower specificity and a 3.9-point lower AUC than reported. The shortfall is concentrated in false positives on NORMAL images and is linked to a train/test distribution

shift, class imbalance and the default threshold. Fine-tuning the top two Inception blocks changed AUC by TBD points and specificity by TBD points, and our pre-registered hypothesis is TBD. Natural next steps are class-weighted loss, choosing the threshold on validation data, X-ray-appropriate augmentation, more seeds, and testing on an external dataset.

References

[1] World Health Organization, "Pneumonia in children," Fact sheet, Nov. 11, 2022. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/pneumonia

[2] D. S. Kermany et al., "Identifying medical diagnoses and treatable diseases by image-based deep learning," Cell, vol. 172, no. 5, pp. 1122–1131, 2018, doi: 10.1016/j.cell.2018.02.010.

[3] P. Rajpurkar et al., "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," arXiv:1711.05225, 2017.

[4] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in Advances in Neural Information Processing Systems 27 (NIPS), 2014, pp. 3320–3328.

[5] M. Raghu, C. Zhang, J. Kleinberg, and S. Bengio, "Transfusion: Understanding transfer learning for medical imaging," in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019.

[6] D. Kermany, K. Zhang, and M. Goldbaum, "Labeled optical coherence tomography (OCT) and chest X-ray images for classification," Mendeley Data, v2, 2018, doi: 10.17632/rscbjbr9sj.2.

[7] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2818–2826.

[8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.

[9] X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, and R. M. Summers, "ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017, pp. 2097–2106.

[10] J. R. Zech, M. A. Badgeley, M. Liu, A. B. Costa, J. J. Titano, and E. K. Oermann, "Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study," PLoS Medicine, vol. 15, no. 11, e1002683, 2018.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[12] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 8024–8035.